

➤ Chapter:-1

❖ What is Node Js:

- Node js is not programming language, it's like a compiler.
- Node js is use to create APIs.
- It is written in C, C++ and JavaScript.
- Chrome's v8 engine is use.
- Node js helps to become full stack developer.

➤ Chapter:-2

❖ First program:

Note : to run node just fire a command “node”.

- Index.js

```
var a=10; var b=20;
console.log(a+b);
```
- Run node file

```
node <file name>
```

➤ Chapter:-3

❖ fundamentals of node js:

- Basic javascript loop ,array, conditions.
- Import form other files.
- Array filter.

- app.js

```
module.exports={  
  x:10,  
  y=20,  
  z:function(){  
    return 10;  
  }  
}
```

- index.js

```
const app = require('./app');  
let a = 10;  
var b = 20;  
const c = 40;  
let arr = [4,65,34,67,23,7];  
let res = arr.filter((item)=>{  
  return item;  
})  
console.log(res);  
console.log(a+b+c);
```

```
console.log(arr);
console.log(app);
for (let i = 0; i < 10; i++)
{
    console.log(i);
}

if(a===10)
{
    console.log("Yes it is 10");
}
else
{
    console.log("No it not 10");
}
```

➤ Chapter:-4

❖ Core modules:

- In every programming language basic features are available like to connect database.
- “console.log()” is core module.
- Some features are we can use without importing is global modules.

- Index.js

```
const fs = require('fs');  
fs.writeFileSync("text.txt","hello file  
system");
```

➤ Chapter:-5

❖ Create server:

- Index.js

```
const http = require('http');  
  
function makeServer(req,resp)  
{  
  resp.write("<h1>Hello server</h1>");  
  resp.end();  
}  
http.createServer(makeServer).listen(4500);
```

➤ Chapter:-6

❖ All about package.json:

- Package.json file contains detail of project. It is heart of project.
- Like name of project and installer package in project.

❖ How to create ?:

- “npm init” is command used.

❖ Install package:

- “npm i simple-node-logger” is command used.

❖ Package-lock.json:

- This file contains details of package installed in project.

➤ **Chapter:-7**

❖ challenge:

- what if my node modules folder deleted ?.
“npm install” use this command
- don't push node_modules folder at GIT.
- Create ‘.gitignore’ file and add folder name like “/node_modules”.

➤ Chapter:-8

❖ All about 'nodemon' package:

- Time saving package.
- When you change code then everytime you need to run command for output.
- It runs project continues.

❖ Install:

- “npm i nodemon -g ”, ' -g' is used to target entire system.

❖ Run using nodemon:

- “nodemon <file_name>”

❖ Interview question:

- Node is a async and single thread.

➤ Chapter:-9

❖ Simple and basic API:

- Index.js

```
const http = require('http');
```

```
http.createServer((req,resp)=>{  
  resp.writeHead(200,{ 'Content-  
Type': 'application\json' });
```

```
  resp.write(JSON.stringify({ name:'test',email:'test  
.com'}));  
  resp.end();  
}).listen(5000);
```

➤ Chapter:-10

❖ Getting input from command line:

- Index.js

```
const fs = require('fs');  
const input = process.argv;  
if(input[2]== 'add')  
{
```

```
        fs.writeFileSync(input[3],input[4]);
    }
    else if(input[2]=='remove')
    {
        fs.unlinkSync(input[3])
    }
    else
    {
        console.log("invalid input");
    }
}
```

➤ Chapter:-11

❖ Display file list from folder:

- Index.js

```
const fs = require('fs');
const path = require('path');
const dirPath = path.join(__dirname,'files');

// for (let i = 0; i < 5; i++) {
//
fs.writeFileSync(dirPath+"/hello"+i+".txt","
s ample file");
// }
```



```
fs.readdir(dirPath,(err,files)=>{  
  files.forEach((item)=>{  
    console.log("file name is ",item);  
  })  
})
```

❖ Interview question:

- Can we access file from other device?
No

➤ Chapter:-12

❖ CRUD with file system:

- Index.js

```
const fs = require('fs');  
const path = require('path');  
const dirPath = path.join(__dirname,'crud');  
const filePath = `${dirPath}/apple.txt`;
```

```
// CREATE FILE  
// fs.writeFileSync(filePath,"This is sample  
file");
```

```
// READ FILE  
// fs.readFile(filePath,'utf8',(err,item)=>{  
//   console.log(item);
```

```
// })

// UPDATE FILE
// fs.appendFile(filePath,'updated
content',(err)=>{
//   if(!err) console.log("file is updated");
// })

// RENAME FILE
//
fs.rename(filePath,`${dirPath}/fruit.txt`,(err)
=>{
//   if(!err)console.log("file name
updated");
// })

// DELETE FILE
// fs.unlinkSync(`${dirPath}/fruit.txt`);
```

❖ Interview question:

- What is buffer?
 - it is temporary memory location.

➤ Chapter:-13

❖ Asynchronous basics:

- Synchronous operation tasks are performed one at time.
- Asynchronous operation not wait to finish tasks.
- Index.js

```
let a = 10;
```

```
let b = 0;
```

```
setTimeout(() => {
```

```
  b=20;
```

```
}, 2000);
```

```
console.log(a+b);
```

➤ Chapter:-14

❖ Handle Asynchronous data:

- For example if any process taking time then we can't get real output.
- For example,setTimeout will take 2 seconds to execute then assign value 30 to variable b.

COSMOS

When we do `console.log(a+b)` then we get output 20, but real output is 50.

- `Index.js`

```
console.log("This is problem");
```

```
let a = 20;
```

```
let b = 0;
```

```
setTimeout(() => {  
  b = 30;  
}, 2000);
```

```
console.log(a+b);
```

```
console.log("This is solution");
```

```
let waitingData = new  
Promise((resolve, reject) => {  
  setTimeout(() => {  
    resolve(30);  
  }, 2000);  
});  
waitingData.then((data) => {  
  b = data;  
  console.log(a+b);  
});
```

➤ Chapter:-15

❖ How node js works:

- Index.js

```
console.log("Start");
```

```
setTimeout(() => {  
    console.log("Wait 2 second");  
}, 2000);
```

```
setTimeout(() => {  
    console.log("Wait 0 second");  
}, 0);
```

```
console.log("Finish");
```

- In above example you will get output like this...
 >start
 >Finish
 >Wait 0 seconds
 >Wait 2 seconds
 ...the reason is node js is build using C, C++,and
 JavaScript, so when you are using function are

based on C and C++ then that functions are call after main() are totally executed.

- Here is the reason when you use setTimeout() that are based on C and C++ it will call after js main().

➤ Chapter:-16

❖ Express js:

- Express js is a framework of node js
- Express js is use to make APIs. You can make API using node js but it is very lengthy process.

❖ How to install express js:

- >>npm install express

❖ Index.js:

```
const express = require('express');  
const app = express();  
app.get("/",(req,res)=>{  
    res.send("This is home page");  
});
```

```
app.get('/about',(req,res)=>{
    res.send("This is about page");
});
app.get('/contact',(req,res)=>{
    res.send("This is contact page");
});
app.listen(5000);
```

➤ Chapter:-17

❖ Routing parameters:

- Follow the example of chapter 16
- Receiving request....
 - ```
app.get('/',(req,res)=>{
 res.send(req.query.name);
});
```
- How to pass request ?  
In the url....  
Localhost:5000/?name=ABC

### ❖ Index.js

```
: const express = require('express');
const app = express();
```

```
app.get("",(req,res)=>{
 res.send(req.query.name);
});
app.listen(5000);
```

## ➤ Chapter:-18

❖ Render html and json:

- Render mean displaying
- Index.js

```
const express = require('express');
const app = express();

app.get("",(req,res)=>{
 res.send("about");
});

app.get("/about",(req,res)=>{
 res.send("home");
});

app.listen(5000);
```



## ➤ Chapter:-19

### ❖ Make html pages:

- Used to create static website that contains home,about,contact etc. pages.
- In previous chapter we render html & json we can't create entire page.
- Create public folder and add html pages like home,about,contact then load url “localhost:5000/about.html ”
- Index.js

```
const express = require('express');
const path = require('path');
```

```
const app = express();
const publicPath =
path.join(__dirname,'public');
```

```
app.use(express.static(publicPath));
```

```
app.listen(5000);
```

## ➤ Chapter:-20

### ❖ Remove extension from url:

- The reason to remove extension from url is for privacy.
- For this example need to create four simple html pages index.html for home , about.html , contact.html and notfound.html.
- Put all file inside public folder.

- Index.js

```
const express = require('express');
const path = require('path');
const app = express();
const publicPath =
path.join(__dirname, 'public');
app.get("", (req, res) => {
 res.sendFile(`${publicPath}/index.html`);
});
app.get('/about', (req, res) => {
 res.sendFile(`${publicPath}/about.html`);
});
```

```
app.get('/contact',(req,res)=>{
 res.sendFile(`${publicPath}/contact.html`);
});
app.get('*', (req,res)=>{

 res.sendFile(`${publicPath}/notfound.html`)
 ;
});
app.listen(5000);
```

## ➤ Chapter:- 21

### ❖ EJS template engine:

- Template engine is use to create dynamic pages.
- To use EJS template,need to install node package.

### ❖ What is dynamic pages ?

- It means contents are received from server or database.
- There are many template engine but main reason to use EJS template is it's more popular and large community.

- Create a folder name is 'views' and create a file name 'profile.ejs'

## ❖ Profile.ejs

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport"
content="width=device-width, initial-scale=1.0">
 <title>Profile</title>
</head>
<body>
 <h1>Welcome <%= user.name%></h1>
 <h2>Welcome <%= user.email%></h2>
 <h3>Welcome <%= user.city%></h3>
</body>
</html>
```

- Create 'public' folder add simple index.html which contain simple heading.

## ❖ Index.js

```
const express = require('express');
const path = require('path');
```

# COSMOS

---

```
const app = express();
const publicPath =
path.join(__dirname,'public');

app.set('view engine','ejs');

app.get('/',(req,res)=>{
 res.sendFile(`${publicPath}/index.html`);
});

app.get('/profile',(req,res)=>{
 const user = {
 name:'demo',
 email:'demo@gmail.com',
 city:'demo'
 }
 res.render(`profile`,{user});
});

app.listen(5000);
```

## ➤ Chapter:-22

### ❖ Dynamic page:

- Index.js

```
const express = require('express');
const path = require('path');
const app = express();
const publicPath =
path.join(__dirname,'public');

app.set('view engine','ejs');

app.get('/',(req,res)=>{
 res.sendFile(`${publicPath}/index.html`);
});

app.get('/profile',(req,res)=>{
 const user = {
 name:'demo',
 email:'demo@gmail.com',
 city:'demo',
 skills:['php','css','js']
 }
 res.render(`profile`,{user});
});
```

# COSMOS

---

```
app.get('/login',(req,res)=>{
 res.render(`login`);
});
```

```
app.listen(5000);
```

- Profile.ejs

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport"
content="width=device-width, initial-
scale=1.0">
 <title>Profile</title>
</head>
<body>
 <%- include('common/header');%>
 <h1>Welcome <%= user.name%></h1>
 <h2>Welcome <%= user.email%></h2>
 <h3>Welcome <%= user.city%></h3>

 <% user.skills.forEach((item)=>{%>
 <%= item%>
 <%});%>

```

```

</body>
</html>
```

- Now we are creating a common header file for our website.

1. Create a folder 'public'.
2. Create simple 'index.html' inside public.
3. Create a folder 'views'.
4. Create a folder 'common' inside 'views'.
5. Create a 'header.ejs' in 'common'.
6. Create 'login.ejs' and 'profile.ejs'.

- Login.ejs

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport"
content="width=device-width, initial-
scale=1.0">
 <title>Login</title>
</head>
```



# COSMOS

---

```
<body>
 <%- include('common/header');%>
 <h1>Login</h1>
</body>
</html>
```

- header.ejs

```
<nav>
 This is header
</nav>
```

- profile.ejs

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport"
content="width=device-width, initial-
scale=1.0">
 <title>Profile</title>
</head>
<body>
 <%- include('common/header');%>
 <h1>Welcome <%= user.name%></h1>
 <h2>Welcome <%= user.email%></h2>
```

# COSMOS

---

```
<h3>Welcome <%= user.city%></h3>

 <% user.skills.forEach((item)=>{>%>
 <%= item%>
 <%});%>

</body>
</html>
```

- index.js

```
const express = require('express');
const path = require('path');

const app = express();
const publicPath =
path.join(__dirname, 'public');

app.set('view engine', 'ejs');

app.get('/', (req, res) => {
 res.sendFile(`${publicPath}/index.html`);
});

app.get('/profile', (req, res) => {
 const user = {
```

# COSMOS

---

```
name:'demo',
email:'demo@gmail.com',
city:'demo',
skills:['php','css','js']
}
res.render(`profile`,{user});
});

app.get('/login',(req,res)=>{
 res.render(`login`);
});
app.listen(5000);
```

## ➤ Chapter:-23

### ❖ Middleware:

- We can only use middleware with router.
- Middleware is a function that are used in router.
- To access and modify request and response middleware is used.
- To reason to access & modify and response to authenticate user like user login or not.

- Index.js

```
const express = require('express');
```

# COSMOS

---

```
const app = express();

const reqFilter = (req,res,next)=>{
 if(!req.query.age)
 {
 res.send("Please provide age");
 }
 else if(req.query.age<18)
 {
 res.send("You cannot vote");
 }
 else
 {
 next();
 }
}

app.use(reqFilter);

app.get('/',(req,res)=>{
 res.send("Welcome to my site");
});

app.get('/about',(req,res)=>{
 res.send("This is about us page");
});
app.listen(5000);
```

## ❖ Types of Middleware:

1. Application level
2. Router level
3. Error handler
4. Built in
5. Third party

## ➤ Chapter:-24

### ❖ Router level Middleware:

- In previous chapter we see middleware was applied on entire application.
- It means every pages are based on middleware.
- What if you want to set middleware to specific pages.
- Also we are creating a middleware in separate file.
- Middleware.js

```
module.exports = (req, resp, next) => {
 if (!req.query.age) {
 resp.send("Please provide your age")
 }
}
```

## COSMOS

---

```
else if (req.query.age<18) {
 resp.send("You are under aged")
}
else {
 next();
}
}
```

- index.js

```
const express = require('express');
const reqFilter = require('./middleware');
const app = express();
const route = express.Router();

route.use(reqFilter);

app.get('/',(req,res)=>{ res.send("Welcome
 to Home page");
});

app.get('/users',(req,res)=>{
 res.send("Welcome to User page");
});
```

```
route.get('/about',(req,res)=>{
 res.send("Welcome to About page");
});

route.get('/contact',(req,res)=>{
 res.send("Welcome to Contact page");
});
app.use('/',route);
app.listen(5000);
```

## ➤ Chapter:-25

### ❖ Install mongoDB:

- Download mongoDB community version also download mongoshell.
- Install the setup.
- Choose install type ‘ complete ’.
- After the installation add ‘ bin ’ folder’s path in environment variable.
- Run command in command prompt to check.  
“ mongo ”

## ➤ Chapter:-26

### ❖ mongoDB basic:

- mongoDB is NoSQL database.
- Here we not store data in table we store data in collection.
- data store in object.
- Basic command:
  - >> show dbs (to display all database)
  - >>use <databasename> (to switch specific database)
  - >>db.createCollection('data') (to create collection)
  - >>db.data.drop() (to remove collection 'data')
  - >>show collection (to display all collection from db)
  - >>db.dropDatabase() (to remove specific database)



## ➤ Chapter:-27

### ❖ CRUD operation in MongoDB:

- In mongo compass out database name is 'e-comm' and collection name is product.
- To use 'e-comm' database use command  
>>e-comm

### ❖ Insert data

```
>>Db.products.insertOne({name:'m40',brand:'samsung',price:250,category:'mobile'})
```

### ❖ Display data:

```
>>db.products.find()
```

### ❖ Update data:

```
>>db.products.updateOne({name:"m40"},{$set:{brand:'oppo'}});
```

### ❖ delete data:

```
>>db.products.deleteOne({brand:'oppo'})
```

❖ interview question:

- what is CRUD operation?  
It means create,read,update,delete.

## ➤ Chapter:-28

❖ Connect Node js with MongoDB:

- Install MongoDB driver  
>>npm i mongodb
- Index.js

```
const {MongoClient}
= require('mongodb')
const url=
'mongodb://127.0.0.1:27017'; const
databaseName='e-comm' const
client= new MongoClient(url);
async function getData()
{
 let result = await client.connect();
 db= result.db(databaseName);
 collection = db.collection('products');
 let data = await
collection.find({}).toArray();
```

```
 console.log(data)
 }
 getData();
```

## ➤ Chapter:-29

❖ Read data from MongoDB:

- Index.js

```
//option-1
```

```
const dbConnect=
require('./mongodb');
```

```
dbConnect().then((resp)=>{
 resp.find({name:'m40'}).toArray().then
 ((data)=>{
 console.log(data)
 })
})
```

```
//option-2
```

```
const main=async ()=>{
 let data = await dbConnect();
```

# COSMOS

---

```
data = await
data.find({name:'m40'}).toArray();
console.log(data)
}
main();
```

- mongodb.js

```
const {MongoClient}
= require('mongodb')
const url=
'mongodb://127.0.0.1:27017'; const
databaseName='e-comm' const
client= new MongoClient(url);

async function dbConnect()
{
let result = await client.connect();
db= result.db(databaseName); return
db.collection('products');

}
module.exports= dbConnect;
```

## ➤ Chapter:-30

### ❖ Insert data from MongoDB:

- Insert.js

//continue with chapter 29

```
const dbConnect = require('./mongodb');
```

```
const insertData=async ()=>{
```

```
 let data = await dbConnect();
```

```
 let result = await data.insertMany(
```

```
 [
```

```
 {name:'max
```

```
5',brand:'micromax',price:420,category:'mo
bile'},
```

```
 {name:'max
```

```
6',brand:'micromax',price:520,category:'mo
bile'},
```

```
 {name:'max
```

```
7',brand:'micromax',price:620,category:'mo
bile'},
```

```
]
)
 if(result.acknowledged)
 {
 console.warn("data is inserted")
 }
}
insertData();
```

## ➤ Chapter:-31

### ❖ Update data from MongoDB:

- update.js

```
//continue with chapter 29
const dbConnect= require('./mongodb')

const updateData=async ()=>{
 let data = await dbConnect();
 let result = await data.updateMany(
 { name:'max 5'},
 {
 $set:{name:'max pro 5', price:1000} }
)
}
```

```
 }
)
 console.log(result)
}
updateData();
```

## ➤ **Chapter:-32**

❖ delete data from MongoDB:

- delete.js

```
//continue with chapter 29
const dbConnect = require('./mongodb');

const deleteData=async ()=>{
 let data = await dbConnect();
 let result = await
 data.deleteMany({name:'max 7'})
 console.log(result)
}
deleteData();
```

## ➤ Chapter:-33

### ❖ APIs with MongoDB:

- Here we are creating a simple API that can retrieve data from MongoDB.
- Api.js

```
//continue with chapter 29
const dbConnect = require('./mongodb');
const express = require('express');
const app = express();

app.use(express.json());
app.get('/', async (res, resp) => {
 let data = await dbConnect();
 data = await data.find().toArray();
 resp.send(data);
});

app.listen(5000)
```

### ❖ interview question:

- can we pass 'body' in GET method?  
No.



## ➤ Chapter:-34

❖ Node js post API method:

- Post method is use to submit data to database.
- Api.js  
const dbConnect = require('./mongodb');  
const express = require('express');  
const app = express();

```
app.use(express.json());
app.get('/', async (res, resp) => {
 let data = await dbConnect();
 data = await data.find().toArray();
 resp.send(data);
});
```

```
app.post("/", async (req, resp) => {
 let data = await dbConnect();
 let result = await
data.insertOne(req.body)
 resp.send(result)
})
app.listen(5000)
```

## ➤ Chapter:-35

❖ Node js put API ( update API ):

- Put method is use to update data,we can update data by using POST method but it is not standard method.

- Api.js

```
const dbConnect = require('./mongodb');
const express = require('express');
const app = express();
```

```
app.use(express.json());
app.get('/', async (res, resp) => {
 let data = await dbConnect();
 data = await data.find().toArray();
 resp.send(data);
});
```

```
app.post("/", async (req, resp) => {
 let data = await dbConnect();
 let result = await
data.insertOne(req.body)
 resp.send(result)
```

```
});

app.put("/:name",async (req,res)=>{
 let data = await dbConnect();
 let result = data.updateOne(
 {name:req.params.name},
 {
 $set:{price:999}
 }
);
 res.send({result:"update"});
});

app.listen(5000)
```

## ➤ **Chapter:-36**

❖ Node js delete API method:

- Api.js

```
const dbConnect = require('./mongodb');
const express = require('express');
const mongodb = require('mongodb');
```

# COSMOS

---

```
const app = express();
app.use(express.json());
app.get('/', async (res, resp) => {
 let data = await dbConnect();
 data = await data.find().toArray();
 resp.send(data);
});
app.post("/", async (req, resp) => {
 let data = await dbConnect();
 let result = await
data.insertOne(req.body)
 resp.send(result)
});
app.put("/:name", async (req, res) => {
 let data = await dbConnect();
 let result = data.updateOne(
 {name: req.params.name},
 {
 $set: {price: 999}
 }
);
```

```
res.send({result:"update"});
});
app.delete("/:id",async (req,res)=>{
 console.log(req.params.id); const
 data = await dbConnect(); const
 result = data.deleteOne({
 _id:new
 mongodb.ObjectId(req.params.id)
 });
 res.send(result);
});
app.listen(5000)
```

## ➤ Chapter:-37

❖ Start with mongoose:

- It is a package use to connect MongoDB with node.
- Mongoose provide some advance features like..

## 1. We can't use schema:

- For example in our collection we have id,name,brand and price.
- If we want to restrict to add new key in MongoDB we can't do it.

## 2. We can't define types to variable:

- For example in our collection we have price like 999. But if we add double quotes in price then it converted into string like "999".

➤ Mongoose prevent above situations.

## ❖ Install mongoose:

>>npm i mongoose

- Schema means total number of fields and model is use to connect node js and MongoDB.

# COSMOS

---

- Index.js

```
const mongoose = require('mongoose');
const main = async () => {
 await
 mongoose.connect("mongodb://127.0.0.1:27017/e-comm");

 const ProductSchema = new
 mongoose.Schema({
 name: String
 });

 const ProductsModel =
 mongoose.model('products', ProductSchema);
 let data = new ProductsModel({ name:
 "m8" });
 let result = await data.save();
 console.log(result)
}
main();
```

## ➤ Chapter:-38

### ❖ CRUD with mongoose:

- Here we are continue with chapter 37.
- Index.js

```
const mongoose =
require('mongoose');
```

```
mongoose.connect('mongodb://127.0.
0.1:27017/e-comm');
const productSchema = new
mongoose.Schema({
 name: String,
 price: Number,
 brand: String,
 category: String
});
```

```
const saveInDB = async () =>
 { const Product =
 mongoose.model('products',
 productSchema);
 let data = new Product({
```



# COSMOS

---

```
 name: "max 100",
 price: 200,
 brand: 'maxx',
 category: 'Mobile'
 });
const result = await data.save();
console.log(result);
}

const updateInDB = async () =>
{
 const Product =
mongoose.model('products',
productSchema);
 let data = await Product.updateOne(
 { name: "max 6" },
 {
 $set: { price: 99990, name: 'max
pro 6' }
 }
)
 console.log(data)
}
```

```
const deleteInDB = async () => {
 const Product =
mongoose.model('products',
productSchema);
```

# COSMOS

---

```
 let data = await
 Product.deleteOne({name:'m40'})
 console.log(data);
 }
 const findInDB = async
 ()=>{ const Product =
 mongoose.model('products',
 productSchema);
 let data = await
 Product.find({name:'max pro 5'})
 console.log(data);
 }
 saveInDB();
```

## ➤ Chapter:-39

### ❖ POST API with mongoose:

- Here we are continue with chapter 37.
- config.js

```
const mongoose = require('mongoose');
mongoose.connect("mongodb://127.0.
0.1:27017/e-comm");
```

# COSMOS

---

- index.js

```
const express = require('express');
require("./config");
const Product = require('./product');
const app = express();
```

```
app.use(express.json());
app.post("/create", async (req, resp)
=> {
 let data = new Product(req.body);
 const result = await data.save();
 resp.send(result);
});
```

```
app.listen(5000);
```

- product.js

```
const mongoose =
require('mongoose');
const productSchema=
mongoose.Schema({
 name:String,
 price:Number,
 brand:String,
 category:String
```

```
});
```

```
module.exports=
mongoose.model("products",productS
chema);
```

## ➤ Chapter:-40

❖ GET ,delete and PUT API:

- This chapter is continue with chapter 39.
- config.js

```
const mongoose = require('mongoose');
mongoose.connect("mongodb://127.0.
0.1:27017/e-comm");
```

- product.js

```
const mongoose =
require('mongoose');
const productSchema=
mongoose.Schema({
 name:String,
 price:Number,
```

# COSMOS

---

```
brand:String,
category:String
});
```

```
module.exports=
mongoose.model("products",productS
chema);
```

- index.js

```
const express = require('express');
require("./config");
const Product = require('./product');
const app = express();
```

```
app.use(express.json());
app.post("/create", async (req, resp)
=> {
 let data = new Product(req.body);
 const result = await data.save();
 resp.send(result);
});
```

```
app.get("/list", async (req, resp) => {
 let data = await Product.find();
 resp.send(data);
```

# COSMOS

---

```
 })
 app.delete("/delete/:_id", async (req,
resp) => {
 console.log(req.params)
 let data = await
Product.deleteOne(req.params);
 resp.send(data);
 })

 app.put("/update/:_id", async (req, resp)
=> {
 console.log(req.params)
 let data = await Product.updateOne(
 req.params,
 {$set: req.body}
);
 resp.send(data);
 })

 app.listen(5000)
```

## ➤ Chapter:-41

### ❖ Search API in Node js with MongoDB:

- There are two types of search one is single field and other is multiple field.
- Single search means you are searching a student with its name only.
- Multiple fields means you are searching a student with its name and roll number.
- Note:  
Use config.js and product.js from chapter 42.
- Index.js

```
const express = require('express');
require("./config");
const Product = require('./product');
const app = express();
```

```
app.use(express.json());
```

```
app.get("/search/:key", async
(req, resp) => {
 let data = await Product.find(
 {
```

# COSMOS

---

```
"$or":[

 {name:{$regex:req.params.key}},

 {brand:{$regex:req.params.key}}
]
}
)
resp.send(data);

})

app.listen(5000)
```

## ❖ Interview question:

- Can we use POST or delete method to search instead of GET.  
Yes, but it is not standard method.

## ➤ Chapter:-42

### ❖ Upload file in Node js:

- Here we are using a npm package name is multer.
- Install multer:
- >>npm i multer



❖ Index.js:

```
const express = require('express');
const multer = require('multer');
const app = express();

const upload = multer({
 storage:multer.diskStorage({
 destination:function(req,file,cb){
 cb(null,"uploads");
 },
 filename:function(req,file,cb){
 cb(null,file.fieldname+"-
"+Date.now()+".jpg");
 }
 })
}).single("user_file");

app.post("/upload",upload,(req,res)=>{
 res.send("file upload");
});

app.listen(5000);
```

❖ Interview question:

- Can we upload file without the package multer.  
Yes, but it is lengthy method.

➤ **Chapter:-43**

❖ Os module in Node js:

- This is operation system module use to collect information about operating system.

- Index.js

```
const os = require('os');
```

```
console.log(os.arch());
```

```
console.log(os.freemem()/(1024*1024
*1024));
```

```
console.log(os.totalmem()/(1024*1024
*1024));
```

```
console.log(os.hostname());
```

```
console.log(os.platform());
```

```
console.log(os.userInfo());
```

## ➤ Chapter:-44

### ❖ Event and Event emitter in Node js:

- When you click a button and some action performed by webpage is a event.
- Event emitter is responsible to generate a event.
- Index.js

```
const express = require('express');
const EventEmitter = require('events');
const app = express();
const event = new EventEmitter();
let count = 0;
```

```
event.on('Count API',()=>{
 count++;
 console.log("Event call "+count);
});
```

```
app.get("/",(req,res)=>{
```

# COSMOS

---

```
res.send("API called")
event.emit("Count
});API")
```

```
app.listen(5000);
```

## ➤ Chapter:-45

### ❖ Read-Eval-Print-Loop:

- REPL is command line prompt .
- When you open command prompt in windows then you apply command “ node ” you can perform the operation of node.
- Like arithmetic operation, creating a function and much more.
- “ .editor ” command provides you a feeling to code editor.

## ➤ Chapter:-46

### ❖ Node js connect with MySQL:

- Install MySQL:

---

COSMOS

---

4<sup>th</sup> Floor, Shiva Blessing – 3, Valentine Circle, Waghawadi Road, Bhavnagar.  
8154910092

---

# COSMOS

---

>>npm I mysql

---

COSMOS

4<sup>th</sup> Floor, Shiva Blessing – 3, Valentine Circle, Waghawadi Road, Bhavnagar.  
8154910092

---

- Index.js

```
const mysql= require("mysql");
```

```
const con= mysql.createConnection({
 host:"localhost",
 user:"root",
 password:"",
 database:"test"
});
```

```
con.connect((err)=>{
 if(err)
 {
 console.warn("not connect")
 }
 else
 {
 console.warn("connected!!!")
 }
})
```

## ➤ Chapter:-47

### ❖ Node js GET API MySQL:

- Config.js

```
const mysql = require("mysql");
const con= mysql.createConnection({
 host:'localhost',
 user:'root',
 password:"",
 database:"test"
});
```

```
con.connect((err)=>{
 if(err)
 {
 console.warn("error in
connection")
 }
});
```

```
module.exports =con;
```

- Index.js

```
const express = require("express");
const con = require("./config");
```

```
const app = express();

app.get("/", (req, resp) => {
 con.query("select * from students",
 (err, result) => {
 if (err) { resp.send("error in api") }
 else { resp.send(result) }
 })
});
app.listen("5000")
```

## ➤ Chapter:-48

### ❖ Node js POST API MySQL:

- Index.js

```
const express = require("express");
const con = require("./config");
const app = express();

app.use(express.json());

app.get("/", (req, resp) => {
 con.query("select * from students",
 (err, result) => {
```



## COSMOS

---

```
 if (err) { resp.send("error in api") }
 else { resp.send(result) }
 })
});
```

```
app.post("/",(req,resp)=>{
 const data=req.body;
 con.query("INSERT INTO students
SET?",data,(error,results,fields)=>{
 if(error) throw error;
 resp.send(results)
 })
});
```

```
app.listen("5000");
```

### ➤ Chapter:-49

❖ Node js PUT API with MySQL:

- Index.js

```
const express = require("express");
const con = require("./config");
const app = express();
```

# COSMOS

---

```
app.use(express.json());
```

```
app.get("/", (req, resp) => {
 con.query("select * from students",
 (err, result) => {
 if (err) { resp.send("error in api") }
 else { resp.send(result) }
 })
});
```

```
app.post("/",(req,resp)=>{
 const data=req.body;
 con.query("INSERT INTO students
SET?",data,(error,results,fields)=>{
 if(error) throw error;
 resp.send(results)
 })
});
```

```
app.put("/:id",(req,resp)=>{
 const data=
[req.body.name,req.body.age,req.params.id];
 console.log(data);
 con.query("UPDATE students SET
student_name = ?, age = ? WHERE id
=
?",
```

---

COSMOS

# COSMOS

---

```
data,(error,results,fields)=>{
 if(error) throw error;
 resp.send(results)
})

})

app.listen("5000");
```

## ➤ Chapter:-50

❖ Node js delete API with MySQL:

- Index.js

```
const express = require("express");
const con = require("./config");
const app = express();
app.use(express.json()); app.get("/",
(req, resp) => {
 con.query("select * from students",
(err, result) => {
 if (err) { resp.send("error in api") }
 else { resp.send(result) }
```

# COSMOS

```
 })
 });
 app.post("/",(req,res)=>{ const
 data=req.body;
 con.query("INSERT INTO students
SET?",data,(error,results,fields)=>{
 if(error) throw error;
 resp.send(results)
 })
 });
 app.put("/:id",(req,res)=>{ const data=
[req.body.name,req.body.age,req.param ms.id];
 console.log(data);
 con.query("UPDATE students SET student_name
= ?, age = ? WHERE id =
?",
 data,(error,results,fields)=>{ if(error)
 throw error; resp.send(results)
 })
 });
 app.delete("/:id",(req,res)=>{ const data=
[req.params.id];
 con.query("DELETE FROM students WHERE id
```

COSMOS

## COSMOS

```
= ?",
 data,(error,results,fields)=>{ if(error)
 throw error; resp.send(results)
 })
});
app.listen("5000");
```

### ➤ Chapter-51:

**Objective:** Learn how to store data using local storage in Node.js.

1. Install the node-localstorage package. This is a popular library that allows you to use local storage in a Node.js environment.
2. npm install node-localstorage
3. Create a simple Node.js application that:
  - Stores a user's name and age in local storage.
  - Retrieves and prints the stored data to the console.

### **Instructions:**

- Use localStorage.setItem() to save the user's name and age.
- Use localStorage.getItem() to retrieve and print the data.

```
javascript
const { LocalStorage } = require('node-localstorage');
// Create a local storage instance
const localStorage = new LocalStorage('./scratch');
// Storing data in local storage
localStorage.setItem('name', 'John Doe');
localStorage.setItem('age', '30');
```

## COSMOS

---

```
// Retrieving and printing stored data
const name = localStorage.getItem('name');
const age = localStorage.getItem('age');
console.log(`Name: ${name}`);
console.log(`Age: ${age}`);
```

### ➤ chapter-52:

## Update Data in Local Storage

**Objective:** Learn how to update data in local storage.

1. Write a Node.js script that updates the age of a user in local storage.
  - Initially, store a user's name and age.
  - Then update the user's age in local storage and print the updated data.

### Instructions:

- Use `localStorage.setItem()` to store the initial data.
- Use `localStorage.setItem()` again to update the age.
- Retrieve and print the updated data.

```
javascript
const { LocalStorage } = require('node-localstorage');
// Create a local storage instance
const localStorage = new LocalStorage('./scratch');
// Storing initial data
localStorage.setItem('name', 'John Doe');
localStorage.setItem('age', '30');
// Updating age
localStorage.setItem('age', '31');
```

## COSMOS

---

```
// Retrieving and printing updated data
const name = localStorage.getItem('name');
const updatedAge = localStorage.getItem('age');
console.log(`Name: ${name}`);
console.log(`Updated Age: ${updatedAge}`);
```

### ➤ chapter-53:

#### **Deleting Data from Local Storage**

**Objective:** Learn how to delete data from local storage.

1. Write a script that:

- Stores a user's information in local storage.
- Deletes the user's age from local storage.
- Retrieves and prints the remaining data.

#### **Instructions:**

- Use `localStorage.setItem()` to store the initial data.
- Use `localStorage.removeItem()` to delete the age.
- Retrieve and print the remaining data (e.g., the user's name).

```
javascript
const { LocalStorage } = require('node-localstorage');
// Create a local storage instance
const localStorage = new LocalStorage('./scratch');
// Storing initial data
localStorage.setItem('name', 'John Doe');
localStorage.setItem('age', '30');
// Deleting age from local storage
localStorage.removeItem('age');
// Retrieving and printing remaining data
const name = localStorage.getItem('name');
```

COSMOS

## COSMOS

---

```
const age = localStorage.getItem('age');
console.log(`Name: ${name}`);
console.log(`Age: ${age ? age : 'Age not available'}`);
```

### ➤ chapter-54:

#### **Clear All Data from Local Storage**

**Objective:** Learn how to clear all data from local storage.

1. Write a Node.js script that:

- Stores multiple pieces of data (name, age, and email).
- Clears all data from local storage.
- Prints the status of local storage after clearing it.

#### **Instructions:**

- Use `localStorage.setItem()` to store multiple pieces of data.
- Use `localStorage.clear()` to clear all data.
- Check if data exists after clearing and print the status.

```
javascript
const { LocalStorage } = require('node-localstorage');
// Create a local storage instance
const localStorage = new LocalStorage('./scratch');
// Storing multiple pieces of data
localStorage.setItem('name', 'John Doe');
localStorage.setItem('age', '30');
localStorage.setItem('email', 'john@example.com');
// Clearing all data
localStorage.clear();
// Checking if data exists
const name = localStorage.getItem('name');
const age = localStorage.getItem('age');
```

COSMOS



## COSMOS

---

```
const email = localStorage.getItem('email');
console.log(`Name: ${name}`);
console.log(`Age: ${age}`);
console.log(`Email: ${email}`);
```

### ➤ chapter-55:

#### **Check If Data Exists in Local Storage**

**Objective:** Learn how to check if data exists in local storage before retrieving it.

1. Write a script that:

- Checks if a user's age is stored in local storage.
- If the age is not available, stores it.
- Retrieves and prints the user's age.

#### **Instructions:**

- Use `localStorage.getItem()` to check if the age exists.
- If the data doesn't exist, use `localStorage.setItem()` to store the age.
- Retrieve and print the stored age.

```
javascript
const { LocalStorage } = require('node-localstorage');
```

```
// Create a local storage instance
const localStorage = new LocalStorage('./scratch');
```

```
// Check if the age exists in local storage
let age = localStorage.getItem('age');
```

```
if (!age) {
```

## COSMOS

---

```
// If age doesn't exist, store it
localStorage.setItem('age', '30');
age = localStorage.getItem('age');
}
console.log(`Age: ${age}`);
```